Scalable Recommendation via Distributed Alternating Least Squares on Amazon Reviews 2023

Project Proposal - Group 0
Group members: Riccardo Fragale, Tommaso Tron, Edoardo F.A. de Cal
Email Addresses: fragale — tommsot — efadc @ kth.se

1 Problem Description

The user-item interaction matrix behind a large e-commerce recommender is simultaneously very large and extremely sparse: in the Movies and TV category of Amazon Reviews, 6.5 million users interact with 747800 items, and well below 0.01% of the possible pairs are observed. A dense representation of this matrix is infeasible, and factorising it requires an algorithm that operates directly on the observed entries while distributing both the data and the least-squares solves across workers. This project investigates the systems problem of building such a pipeline end to end: ingesting raw review data into a columnar store, learning latent factors with distributed Alternating Least Squares (ALS), precomputing top-N lists for every user, and serving them from a NoSQL store under a key-lookup access pattern. The central questions are how wall-clock training time responds to input volume, where the pipeline's bottlenecks lie, and what recommendation quality the resulting model achieves.

2 Tools

- **Storage:** raw JSONL is converted to columnar **Parquet**, yearly partitioned, for efficient pruning.
- **Processing:** Apache Spark (PySpark, Spark SQL) for parsing, cleaning and filtering; Spark MLlib (`pyspark.ml.recommendation.ALS`) for training, regression and ranking evaluation.
- **Serving:** precomputed top-20 lists are bulk-written to **MongoDB**, one document per user keyed on `user_id`, holding the ranked item identifiers and scores. A **Streamlit** dashboard queries MongoDB to display per-user recommendations, joined against item titles, alongside evaluation and scalability results.

3 Data

We use **Amazon Reviews 2023**, released by the McAuley Lab at UC San Diego (`https://amazon-reviews-2023.github.io/`), public data from 1996 to 2023. We restrict the study to the **Movies and TV** category as the whole dataset is too big: 17.3 million reviews from 6.5 million users over 747.8 thousand items. Each record provides `user_id`, `parent_asin` (item identifier), `rating` (1–5), `timestamp`, `helpful_vote` and `verified_purchase`; the companion metadata file supplies item titles for the dashboard. Development and hyperparameter search are carried out on a **user-level sample** (a random 20% of users, retaining each sampled user's full history); the selected configuration is then retrained on the complete category, which is the configuration reported as the headline result.

4 Methodology and Algorithm

1. **ETL:** Parse the JSONL into a typed DataFrame, project to the four required columns, drop duplicate (user, item) pairs keeping the most recent, and map string identifiers to dense integers as ALS requires. Each stage writes a Parquet checkpoint so that runs are restartable.
2. **Temporal split and filtering:** Interactions are split by timestamp into training (earliest 80%), validation (next 10%) and test (latest 10%). Iterative k-core filtering is applied to the training window, and the resulting user and item vocabularies are projected onto validation and test.
3. **Model:** A rating is modelled as $\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i$. The global mean and the user and item biases are computed in Spark SQL on the training set; explicit-feedback ALS then factorises the residuals. Grid search over rank and regularization, selected on validation NDCG@10.
4. **Evaluation:** RMSE on held-out ratings, and Precision@10 and NDCG@10 on top-K lists. Four systems are compared: a popularity baseline, bias-only, plain ALS, and bias+ALS.
5. **Scalability:** Wall-clock time per stage is reported at 25%, 50% and 100% of the input at fixed resources, isolating how each stage responds to volume.
6. **Serving:** Top-20 lists are generated in batches via `recommendForUserSubset` over chunks of users, bounding peak memory, and bulk-written to MongoDB.